# Programación por bloques

Nombre: _____ Fecha: _____

## Qué vas a aprender

- Entender qué es un algoritmo y representarlo con diagramas de flujo
- Conocer el entorno de programación por bloques y sus elementos: objetos, eventos y scripts
- Usar secuencias, bucles, condicionales, variables y operadores
- Diseñar, probar y depurar programas sencillos, como juegos y animaciones

## Para pensar

**Un videojuego sencillo puede tener cientos de órdenes. ¿Cómo sabe el ordenador cuándo debe ejecutar cada una?**

## Explicación

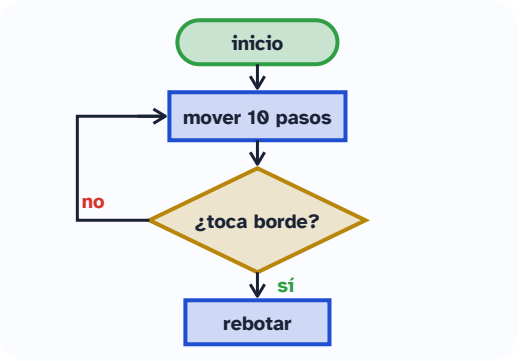
### Algoritmos y programas

Un algoritmo es una secuencia **ordenada y finita** de instrucciones claras para resolver un problema o realizar una tarea. Una receta de cocina, las instrucciones de montaje de un mueble o los pasos para resolver una ecuación son algoritmos. Un programa es un algoritmo escrito en un **lenguaje de programación** que el ordenador puede ejecutar. Los ordenadores no piensan: hacen **exactamente** lo que se les ordena, en el orden indicado, por eso las instrucciones deben ser precisas y sin ambigüedades. Programar es descomponer un problema grande en pasos pequeños, una habilidad llamada **pensamiento computacional**, útil mucho más allá de la informática.

undefined

### Diagramas de flujo

Antes de programar conviene representar el algoritmo de forma gráfica con un diagrama de flujo. Usa símbolos normalizados unidos por flechas que indican el orden: el **óvalo** marca el **inicio** y el **fin**; el **rectángulo**, una **acción** o proceso («mover 10 pasos»); el **rombo**, una **decisión** con una pregunta de respuesta sí/no («¿toca el borde?»), del que salen dos caminos; y el **paralelogramo**, una **entrada o salida** de datos («pedir el nombre», «mostrar resultado»). Si una flecha vuelve hacia atrás, el diagrama tiene un **bucle**. Así se detectan errores de lógica antes de escribir el código.




### La programación por bloques

En la programación por bloques, las instrucciones son piezas de colores que se **encajan** como un puzle, en lugar de escribirse con texto. Así no hay errores de escritura y es fácil ver la estructura del programa. El entorno más conocido es **Scratch**, creado por el MIT; otros son **MakeCode** (para micro:bit), **mBlock** o **Arduinoblocks** (para robots), **App Inventor** (para aplicaciones de móvil) y **Snap!**. En Scratch hay un **escenario**, donde ocurre la acción; unos **objetos** o *sprites*, los personajes, cada uno con sus **disfraces** y sonidos; y para cada objeto, unos **programas** o *scripts* formados por bloques. Los bloques se agrupan por **categorías** de colores: movimiento, apariencia, sonido, eventos, control, sensores, operadores y variables.

undefined

## Eventos y secuencias

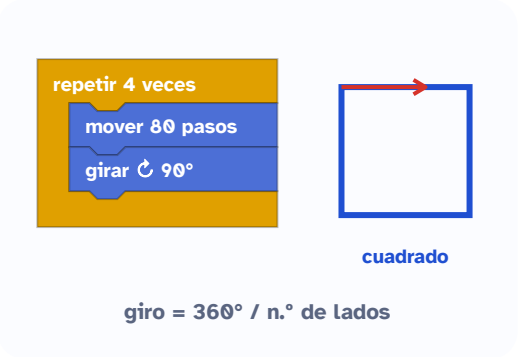
Un programa por bloques empieza siempre con un evento: un bloque con forma de «sombrero» que indica **cuándo** se ejecuta el script. Por ejemplo: «al hacer clic en la bandera verde», «al presionar la tecla espacio», «al hacer clic en este objeto» o «al recibir un mensaje». Debajo del evento se encajan las instrucciones, que se ejecutan en **secuencia**, de arriba abajo, una tras otra. El orden importa: no es lo mismo «girar 90° y mover 50 pasos» que «mover 50 pasos y girar 90°». Varios scripts pueden ejecutarse **a la vez** (en paralelo), cada uno con su propio evento, y los objetos se pueden comunicar entre sí **enviando mensajes**.




## Bucles: repetir instrucciones

Un bucle repite un grupo de instrucciones varias veces, sin tener que copiarlas. Hay tres tipos principales. «**Repetir N veces**» ejecuta el bloque un número fijo de veces: para dibujar un cuadrado basta con «repetir 4: mover 100, girar 90°». «**Por**

**siempre**» repite sin fin, mientras el programa esté en marcha: se usa para comprobar continuamente si se pulsa una tecla o si el personaje toca algo. «**Repetir hasta que**» repite mientras no se cumpla una condición: «repetir hasta que toque la meta». Los bucles hacen los programas más **cortos**, más **fáciles de leer** y de modificar. Para dibujar un polígono de n lados, el ángulo de giro es **360° / n**.




## Condicionales: tomar decisiones

Un condicional permite que el programa **decida** qué hacer según se cumpla o no una condición. El bloque «**si … entonces**» ejecuta sus instrucciones solo si la condición es verdadera: «si toca el color rojo, entonces volver al inicio». El bloque «**si … entonces … si no**» tiene dos caminos: uno si se cumple y otro si no. Las condiciones se construyen con bloques de **sensores** (¿tocando el ratón?, ¿tecla presionada?, ¿tocando a otro objeto?) y de **operadores**: de comparación (**>, <, =**) y lógicos (**y**, **o**, **no**). Un condicional dentro de un bucle «por siempre» es la base de casi todos los videojuegos: comprueba sin parar lo que ocurre y reacciona.

undefined

## Variables y operadores

Una variable es un espacio de la memoria con un **nombre** donde el programa guarda un **dato que puede cambiar**: la puntuación, las vidas, el tiempo, el nombre del jugador. Con los bloques de variables se puede **crear** una variable, **darle un valor** («dar a puntos el valor 0»), **cambiarla** («sumar 1 a puntos») y **mostrarla** en el escenario. Los **operadores** realizan cálculos (suma, resta, multiplicación, división, número aleatorio entre 1 y 10), comparan valores y unen textos. Por ejemplo, para un juego de preguntas: «preguntar ¿cuánto es 7 × 8? y

esperar»; «si respuesta = 56, entonces sumar 1 a puntos». Las **listas** guardan muchos datos en una sola variable.




## Diseñar un videojuego

Para crear un programa más grande, como un **videojuego**, conviene seguir el proceso tecnológico. Primero, **definir** la idea: de qué trata, cuál es el objetivo, cómo se gana y cómo se pierde. Segundo, **descomponer** el problema en partes: el movimiento del personaje con las flechas, los enemigos que se mueven solos, la puntuación, las vidas, el fin del juego. Tercero, **programar** cada parte por separado y probarla. Cuarto, **unir** las partes y comunicar los objetos con mensajes. Por último, **probar** con otras personas y mejorar. Programar bien también es escribir programas ordenados: dar nombres claros a las variables y objetos y crear **bloques propios** (funciones) para las tareas que se repiten.

undefined

## Errores y depuración

Es normal que un programa no funcione a la primera. Los fallos se llaman errores o *bugs*, y el proceso de encontrarlos y corregirlos, **depuración**. Hay errores de **secuencia** (instrucciones en el orden incorrecto), de **lógica** (una condición mal planteada, por ejemplo «>» en lugar de «<»), de **bucles infinitos** que nunca terminan y de **inicialización** (olvidar poner la puntuación a 0 al empezar). Estrategias para depurar: ejecutar el programa **paso a paso**; **mostrar las variables** en pantalla para ver cómo cambian; hacer que el personaje «diga» valores intermedios; probar cada parte por separado; y explicar el programa en voz alta a otra persona, que a menudo revela el fallo.

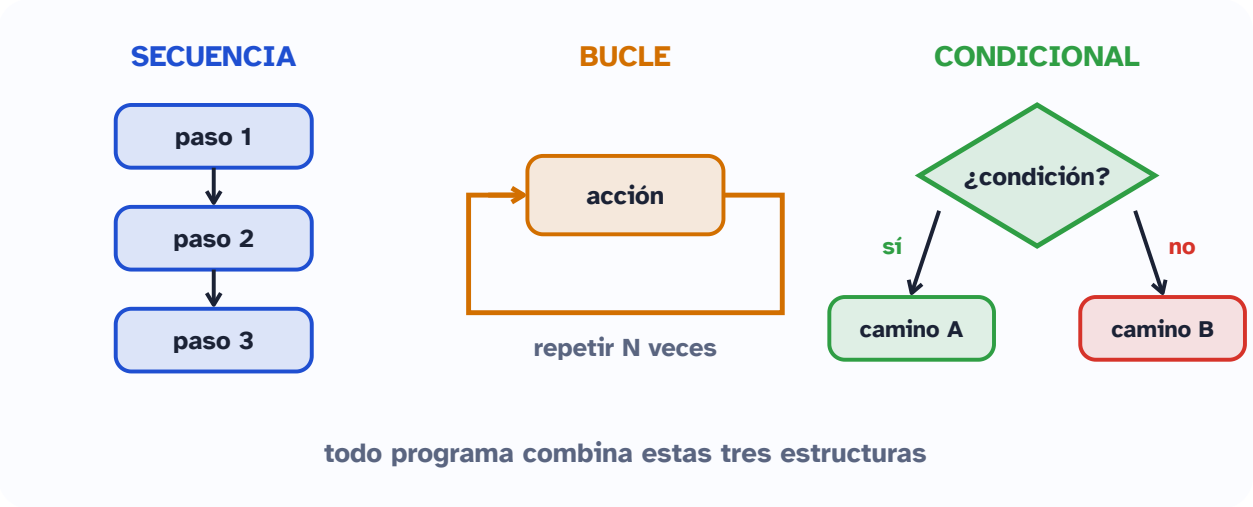
undefined

## De los bloques al código

La programación por bloques es la puerta de entrada a los **lenguajes de texto**, como **Python**, **JavaScript** o **C++**, que usan las mismas ideas: secuencias, bucles, condicionales, variables y funciones. Muchos entornos, como MakeCode o mBlock, permiten ver el **código equivalente** de un programa de bloques. Por ejemplo, «repetir 4 veces: mover 100, girar 90°» en Python sería un bucle `for i in range(4):`. La programación por bloques también sirve para controlar **robots** y placas como **micro:bit** o **Arduino**, que veremos en el tema de robótica, y para crear **aplicaciones de móvil** con App Inventor.

undefined

## Las tres estructuras de control



## Ejercicios

1. **Define algoritmo y pon un ejemplo de la vida cotidiana.**

2. **Dibuja los símbolos del diagrama de flujo para inicio, acción y decisión.**

3. **Nombra cuatro eventos que pueden iniciar un script.**

4. **¿Qué tres tipos de bucle hay en Scratch?**

5. **¿Qué ángulo hay que girar para dibujar un hexágono?**

6. **¿Qué diferencia hay entre «si … entonces» y «si … entonces … si no»?**

7. **¿Qué es una variable? Da tres ejemplos en un juego.**

8. **Escribe un programa que pregunte «¿cuánto es 6 × 7?» y diga «¡Bien!» o «Inténtalo otra vez».**

9. **Nombra tres tipos de errores de programación.**

10. **Cita dos entornos de programación por bloques y para qué sirven.**

## Practica

1. **Una secuencia ordenada y finita de instrucciones es un...**
   - **A)** algoritmo
   - **B)** sprite
   - **C)** evento
2. **En un diagrama de flujo, la decisión se dibuja con un...**
   - **A)** rectángulo
   - **B)** rombo
   - **C)** óvalo
3. **Scratch fue creado por el...**
   - **A)** MIT
   - **B)** CERN
   - **C)** Google
4. **Los personajes de Scratch se llaman...**
   - **A)** objetos (sprites)
   - **B)** variables
   - **C)** escenarios
5. **El bloque con forma de sombrero es un...**
   - **A)** evento
   - **B)** bucle
   - **C)** operador
6. **Para repetir siempre se usa...**
   - **A)** repetir 10
   - **B)** por siempre
   - **C)** si ... entonces
7. **Para dibujar un pentágono se gira...**
   - **A)** 72°
   - **B)** 60°
   - **C)** 90°
8. **Un dato que cambia durante el programa se guarda en una...**
   - **A)** variable

**B)** constante

**C)** disfraz

9. **El operador «y» exige que...**

**A)** se cumpla una condición

**B)** se cumplan las dos

**C)** no se cumpla ninguna

10. **Buscar y corregir errores se llama...**

**A)** compilar

**B)** depurar

**C)** ejecutar

## Explícalo con tus palabras

Incluye: Qué es un algoritmo y cómo se dibuja un diagrama de flujo; La diferencia entre un bucle y un condicional; Para qué sirve una variable en un juego.

## Resumen



## Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

**Algoritmo**

Secuencia ordenada y finita de instrucciones.

**Diagrama de flujo**

Óvalo, rectángulo, rombo y flechas.

**Evento**

Lo que hace empezar un script.

**Bucle**

Repite instrucciones: N veces, siempre o hasta que.

**Condicional**

Si … entonces … si no.

**Variable**

Dato con nombre que puede cambiar.

**Polígono**

Giro = 360° / n.º de lados.

**Depurar**

Encontrar y corregir errores.

## Soluciones

1. **A)** Algoritmo.
2. **B)** Rombo.
3. **A)** El MIT.
4. **A)** Objetos.
5. **A)** Evento.
6. **B)** Por siempre.
7. **A)** 360 / 5 = 72°.
8. **A)** Variable.
9. **B)** Las dos.
10. **B)** Depurar.

## Soluciones de los ejercicios

1. Secuencia ordenada y finita de instrucciones para resolver un problema; una receta o lavarse los dientes.
2. Óvalo, rectángulo y rombo.
3. Clic en la bandera, presionar una tecla, clic en el objeto, recibir un mensaje.
4. Repetir N veces, por siempre y repetir hasta que.
5. 360° / 6 = 60°.
6. El primero solo actúa si se cumple la condición; el segundo tiene también un camino para cuando no se cumple.
7. Un espacio de memoria con nombre que guarda un dato que cambia; puntos, vidas, tiempo.
8. Preguntar y esperar; si respuesta = 42 entonces decir «¡Bien!» si no decir «Inténtalo otra vez».
9. De secuencia, de lógica, bucles infinitos (o de inicialización).
10. Scratch (juegos y animaciones) y MakeCode (micro:bit) o App Inventor (apps de móvil).